

LOVELY PROFESSIONAL UNIVERSITY

ACADEMIC TASK 2

CSE 316

(OPERATING SYSTEM)

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Name: Lovepreet Singh

Registration number: 12411255

Roll No.: 17

Section: K24PR

Submitted to:

Mr. Kundan

DECLARATION

We, **Piyush Dhami**, **Lovepreet Singh** and **Vennam Siddu** students of **Bachelor of Technology in Computer Science and Engineering** at **Lovely Professional University, Punjab**, hereby declare that the project titled **"Real-Time Multithreaded Application Simulator"** is the result of our collective and original work.

We affirm that the work presented in this report has been carried out by us independently and has not been submitted, wholly or in part, to any other institution or university for the award of any degree, diploma, or certification. All information, data, figures, and references used in the project have been properly acknowledged in accordance with academic standards.

We take full responsibility for the accuracy, authenticity, and integrity of the content presented in this report. The project has been completed under the guidance and supervision of faculty members at Lovely Professional University.

Date:
Place: Lovely Professional University, Punjab

Signatures of Candidates:

1. **Piyush Dhami**
   Registration Number: 12411498
2. **Lovepreet Singh**
   Registration Number: 12411255
3. **Vennam Siddu**
   Registration Number: 12415024

PROJECT REPORT

Real-Time Multithreaded Application Simulator

1. Project Overview

The **Real-Time Multithreaded Application Simulator** is an interactive, web-based educational platform developed to demonstrate the internal operational mechanisms of modern operating systems. The primary objective of this simulator is to visually depict how operating systems manage multithreading, CPU scheduling, thread-to-core mapping, and synchronization processes in real time, thereby bridging the gap between theoretical concepts and practical understanding.

In contemporary computing environments, multithreading plays a crucial role in enhancing system performance, resource utilization, and responsiveness. However, the abstract nature of thread scheduling, state transitions, and synchronization often makes these concepts difficult for learners to grasp. This simulator addresses that challenge by providing a dynamic visualization of:

- Thread life cycles (NEW, READY, RUNNING, BLOCKED, TERMINATED)
- CPU scheduling algorithms such as Round Robin, FCFS, and Priority Scheduling
- Thread-to-core mapping models including Many-to-One, One-to-One, and Many-to-Many
- Synchronization primitives like semaphores and monitors

The backend of the simulator is implemented using **Python (Flask)**, which manages the simulation logic, state transitions, and scheduling workflow. The frontend, built with **HTML, CSS, and JavaScript**, provides an intuitive dashboard that updates in real time, enabling users to monitor thread movement, CPU activity, scheduling decisions, and blocking events.

This project serves multiple academic and practical purposes:

1. It enhances conceptual clarity for students studying operating system principles.
2. It acts as a teaching aid for instructors explaining concurrency and synchronization.
3. It enables experimentation with scheduling policies and synchronization mechanisms.

4. It demonstrates real-world system behavior in a controlled, visualized environment.

Overall, the Real-Time Multithreaded Application Simulator is a comprehensive instructional tool that transforms complex OS functionalities into an interactive and accessible learning experience.

2. Module-Wise Breakdown

2.1 Backend (Flask Application Layer)

The backend of the Real-Time Multithreaded Application Simulator is implemented using **Python and the Flask microframework**, serving as the central execution environment where all simulation logic, concurrency modeling, and scheduling computations take place. It operates as the authoritative controller of system state, managing a unified Simulation Controller instance that encapsulates thread management, kernel thread allocation, scheduling workflows, synchronization primitives, and performance metrics.

The backend is architected around a **continuous simulation loop**, executed inside a dedicated background thread. This loop represents the system clock and is responsible for advancing the simulation at fixed intervals, known as ticks. During each tick, the backend performs a sequence of coordinated operations:

1. Evaluates runnable threads according to the selected scheduling algorithm (Round Robin, FCFS, or Priority).
2. Assigns eligible threads to available kernel threads based on the configured mapping model (Many-to-One, One-to-One, or Many-to-Many).
3. Executes one quantum of computation for each running thread, updating its remaining execution time.
4. Detects events such as thread completion, blocking on synchronization operations, or preemption due to quantum expiration.
5. Performs context switches and updates thread states accordingly.

All backend operations are **thread-safe and concurrency-controlled**, employing re-entrant locks (RLocks) to prevent race conditions between the simulation loop and incoming REST API requests. This ensures that thread tables, scheduler queues, and synchronization objects remain consistent even while the frontend interacts with the system in real time.

The backend exposes an extensive set of REST API endpoints that provide fine-grained control over simulation behavior. These endpoints support operations such as dynamic thread creation, modification of scheduler and mapping configurations, execution of WAIT/SIGNAL in semaphores, handling of monitor wait/signal/broadcast actions, and retrieval of a complete snapshot of the current simulation state. Each API request is processed synchronously to maintain deterministic behavior, while state updates are serialized and transmitted to the frontend at regular intervals.

Furthermore, the backend encapsulates implementations of core synchronization constructs, including counting semaphores and monitor-style condition variables. These constructs are integrated seamlessly into the simulation loop, ensuring that blocking, unblocking, and wake-up events occur precisely as they would in a real operating system.

Overall, the backend functions as a **fully autonomous scheduling and synchronization engine**, faithfully modeling OS-level thread execution while providing real-time communication with the presentation layer. Its modular architecture and strong separation of concerns allow for extensibility, maintainability, and high-fidelity simulation of concurrent systems.

2.2 Simulation Controller

The **Simulation Controller** is the central coordinating component of the Real-Time Multithreaded Application Simulator. It acts as the core engine responsible for managing all operational aspects of the simulation, including thread lifecycle management, scheduling, kernel thread assignment, synchronization handling, and system-wide state updates. Every activity inside the simulator—whether triggered by the backend simulation loop or by a frontend API request—is directed and processed through the Simulation Controller.

The controller maintains the **global simulation state**, which includes:

- A table of all active, blocked, and completed threads
- A pool of kernel threads representing available CPU cores
- The currently selected scheduling algorithm (Round Robin, FCFS, or Priority)
- The chosen thread-to-core mapping model (Many-to-One, One-to-One, Many-to-Many)
- Lists of synchronization primitives such as semaphores and monitors
- Simulation metrics including tick count, context switches, and completed thread count

At the heart of the controller is the `_run_tick()` method, executed once every simulation cycle. During each tick, the controller performs the following operations:

1. **Thread Selection:**
   It evaluates the READY queue and selects the next thread to run based on the active scheduling algorithm.
2. **Kernel Assignment:**
   The chosen thread is assigned to a kernel thread according to the configured mapping

policy.
For example:
    - Many-to-One maps all user threads to a single kernel thread
    - One-to-One gives each thread its own kernel
    - Many-to-Many dynamically distributes threads across multiple kernels
3. **Thread Execution:**
   The controller simulates execution by allowing the thread to run for one quantum, reducing its remaining burst time accordingly.
4. **State Transitions:**
   Based on execution outcomes, the controller updates the thread's state:
    - If remaining time becomes zero → TERMINATED
    - If a synchronization operation blocks the thread → BLOCKED
    - Otherwise → remains RUNNING or returns to READY (Round Robin)
5. **Context Switching:**
   If a thread transition requires replacing the running thread with another, the controller records a context switch, enhancing the realism of OS scheduling behavior.
6. **Synchronization Handling:**
   The controller coordinates semaphore WAIT/SIGNAL operations and monitor wait/signal/broadcast actions, ensuring threads are correctly blocked or unblocked and reinserted into scheduler queues.
7. **State Reporting:**
   At the end of each tick, the controller prepares a complete simulation snapshot and returns it to the backend API layer, which forwards it to the frontend for visualization.

All controller operations are designed to be **thread-safe**, using locking mechanisms to prevent race conditions between the simulation loop and concurrent API requests.

Overall, the Simulation Controller serves as the **decision-making core** of the simulator. It governs how threads behave, how CPUs execute tasks, how scheduling policies are enforced, and how synchronization affects execution flow. Without this component, the system would not be able to replicate realistic operating system behavior.

Role in the Simulation

The Synchronization Module integrates tightly with the Simulation Controller:

- When a thread becomes blocked due to WAIT or monitor wait, the controller removes it from its kernel thread and scheduler queue.
- When SIGNAL or monitor notify is triggered, the module reactivates threads, pushing them back into READY state.
- This behavior is reflected immediately in the frontend, showing realistic thread blocking queues and wake-up sequences.

2.3 Frontend

The frontend of the Real-Time Multithreaded Application Simulator is designed as an interactive and visually intuitive interface that allows users to observe and control all aspects of the simulation in real time. It is developed using HTML5, CSS3, and JavaScript, forming a lightweight yet responsive client-side layer that communicates continuously with the backend through REST API calls.

The primary purpose of the frontend is to translate complex operating system behaviors into easy-to-understand visual components. To achieve this, the interface is structured into multiple panels, each representing different elements of the simulation such as thread tables, CPU cores, synchronization primitives, and event logs. The frontend polls the backend at regular intervals (typically every 500 ms) to fetch the latest simulation state and updates the display dynamically based on the received data.

Key responsibilities of the frontend include:

1. Real-Time Visualization
    - Displays all threads with attributes such as ID, name, priority, remaining burst time, and state (NEW, READY, RUNNING, BLOCKED, TERMINATED).
    - Shows CPU core assignments and transitions as threads move through different stages of execution.
    - Provides clear visual indicators for blocked threads in semaphores or monitor wait queues.
2. User Interaction and Controls
   The frontend provides an intuitive control panel enabling users to:
    - Start, pause, resume, or reset the simulation
    - Execute the simulation one tick at a time (step mode)
    - Change scheduling algorithms and mapping models
    - Adjust quantum values
    - Create new threads dynamically by entering burst time and priority
3. Synchronization Interface
    - Lets users create semaphores or monitors directly from the UI
    - Provides buttons for WAIT, SIGNAL, and monitor operations
    - Displays blocked queues in real time, helping users visualize synchronization delays and interactions
4. Event Logging System
   A dedicated log panel records key simulation events such as:
    - Thread creation
    - State transitions (READY → RUNNING → BLOCKED → TERMINATED)
    - Context switches
    - Semaphore WAIT/SIGNAL events
    - Monitor wait/signal operations

This log acts as a real-time explanation of everything happening inside the simulation engine.

Overall, the frontend acts as the visual and interactive layer of the simulator. While the backend performs all scheduling and synchronization logic, the frontend ensures users can understand and manipulate the simulation effortlessly. Together, they create a powerful tool for learning and experimenting with operating system multithreading behavior.

---

3. Functionalities

The Real-Time Multithreaded Application Simulator provides a comprehensive set of functionalities designed to model the behavior of multithreaded systems as implemented in modern operating systems. These functionalities enable users to visualize scheduling decisions, thread execution, synchronization effects, and system-level interactions in real time. The major functionalities include:

3.1 Thread Management

- Allows dynamic creation of new threads by specifying attributes such as thread name, CPU burst time, and priority.
- Displays thread lifecycle transitions, including NEW, READY, RUNNING, BLOCKED, and TERMINATED states.
- Updates thread status in real time as scheduling and synchronization events occur.

3.2 Scheduling Algorithms

The simulator supports multiple CPU scheduling algorithms, enabling comparative study of their behaviors:

- **Round Robin (RR):** Time-sliced, fair scheduling for all threads.
- **First-Come-First-Served (FCFS):** Non-preemptive scheduling based on arrival order.
- **Priority Scheduling:** Selects threads with the highest priority, with tie-breaking handled by thread ID.

Users can switch between algorithms at runtime, allowing observation of how different schedulers impact context switching and thread execution order.

3.3 Thread-to-Core Mapping Models

The simulator demonstrates three thread-to-kernel mapping strategies used in OS design:

- **Many-to-One:** All user threads mapped to a single kernel thread.
- **One-to-One:** Each user thread directly mapped to its own kernel thread.
- **Many-to-Many:** Multiple user threads dynamically distributed across multiple kernel threads.

These models help users understand the relationship between user-level and kernel-level threads.

3.4 Real-Time Visualization

- Live updating thread table showing IDs, priorities, burst times, remaining times, states, and CPU assignments
- Real-time CPU core visualization showing which thread is executing on which core
- Visual representation of blocked queues for semaphores and monitors
- Continuous visual feedback on how the scheduling algorithm and synchronization operations affect thread execution

These metrics help in performance comparison between different schedulers and configurations.

---

4. Technology Used

The development of the Real-Time Multithreaded Application Simulator incorporates a combination of programming languages, libraries, and development tools. These technologies collectively enable real-time simulation, interactive visualization, and efficient backend–frontend communication.

4.1 Programming Languages

Python
Used for implementing the core simulation engine, backend logic, scheduling algorithms, synchronization mechanisms, and REST API layer. Its extensive standard library and threading capabilities make it suitable for constructing multithreading models and simulation workflows.

JavaScript
Responsible for handling frontend interactivity, real-time UI updates, state polling, event logging, and user controls. JavaScript enables seamless communication with the Flask backend via asynchronous fetch requests.

HTML5 / CSS3
Used to design and structure the user interface. HTML provides the layout for system panels such as the thread table, CPU core display, and synchronization modules, while CSS (including modern design patterns like glassmorphism) ensures a clean, responsive, and visually engaging presentation.

4.2 Libraries and Tools

Flask 2.2.5
A lightweight Python web framework used to build the REST API that mediates communication between the frontend and the simulation engine. Flask enables efficient handling of user actions such as starting, pausing, stepping, and resetting the simulation.

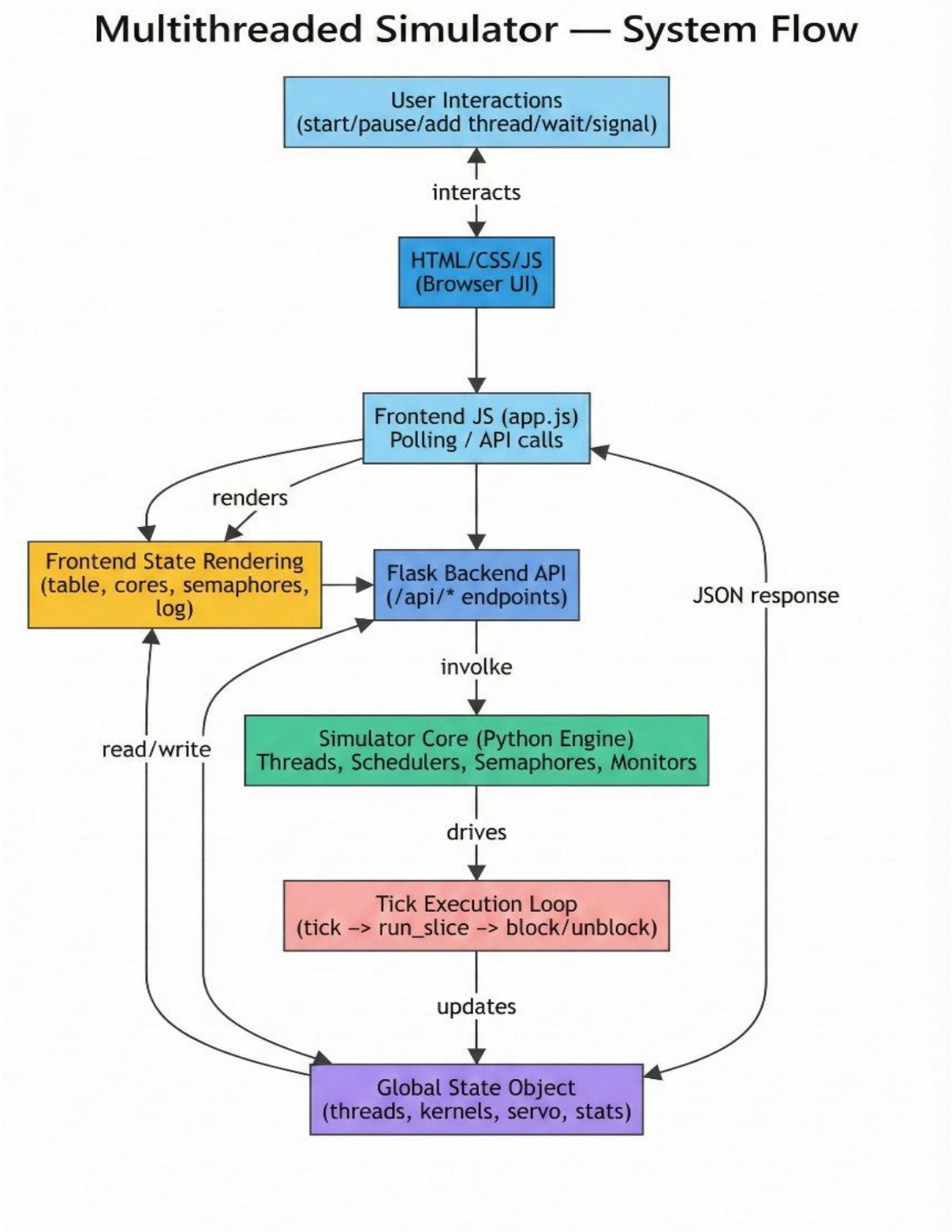
Python Threading Module
Used to implement the background simulation loop and ensure concurrency within the backend. Enables simultaneous handling of API requests and state progression.

4.3 Other Tools

GitHub
Utilized for version control, repository hosting, collaboration, and tracking updates throughout the development lifecycle.

Python Virtual Environment (venv)
Ensures dependency isolation and consistent runtime environments across development and deployment machines.

5. Flow Diagram :



6. Revision Tracking on GitHub

Repository Name: Multithreaded-simulator
GitHub Link: https://github.com/Lovepreetsingh2006/multithreaded-simulator

---

7. Conclusion and Future Scope

Conclusion

The Real-Time Multithreaded Application Simulator successfully demonstrates the internal workings of operating system–level multithreading through an interactive and visually rich interface. By integrating scheduling algorithms, thread-to-core mapping models, and synchronization primitives such as semaphores and monitors, the simulator provides a comprehensive platform for understanding core OS concepts that are otherwise abstract and difficult to visualize.

The backend effectively models realistic thread behavior—such as context switching, blocking, preemption, and execution progress—while the frontend translates these operations into intuitive visual components. This dual-layer architecture enhances both clarity and engagement, making the system an effective learning tool for students, educators, and developers studying concurrent programming and operating system design.

Through real-time updates, user-driven configuration, and detailed event logs, the simulator enables experimentation with various scheduling strategies and synchronization constructs. As a result, it serves as both an educational solution and a practical demonstration of how modern operating systems manage concurrency, CPU allocation, and thread lifecycle transitions.

Future Scope

The current system provides a strong foundation for multithreading simulation, but there is considerable scope for enhancement and expansion. Potential future improvements include:

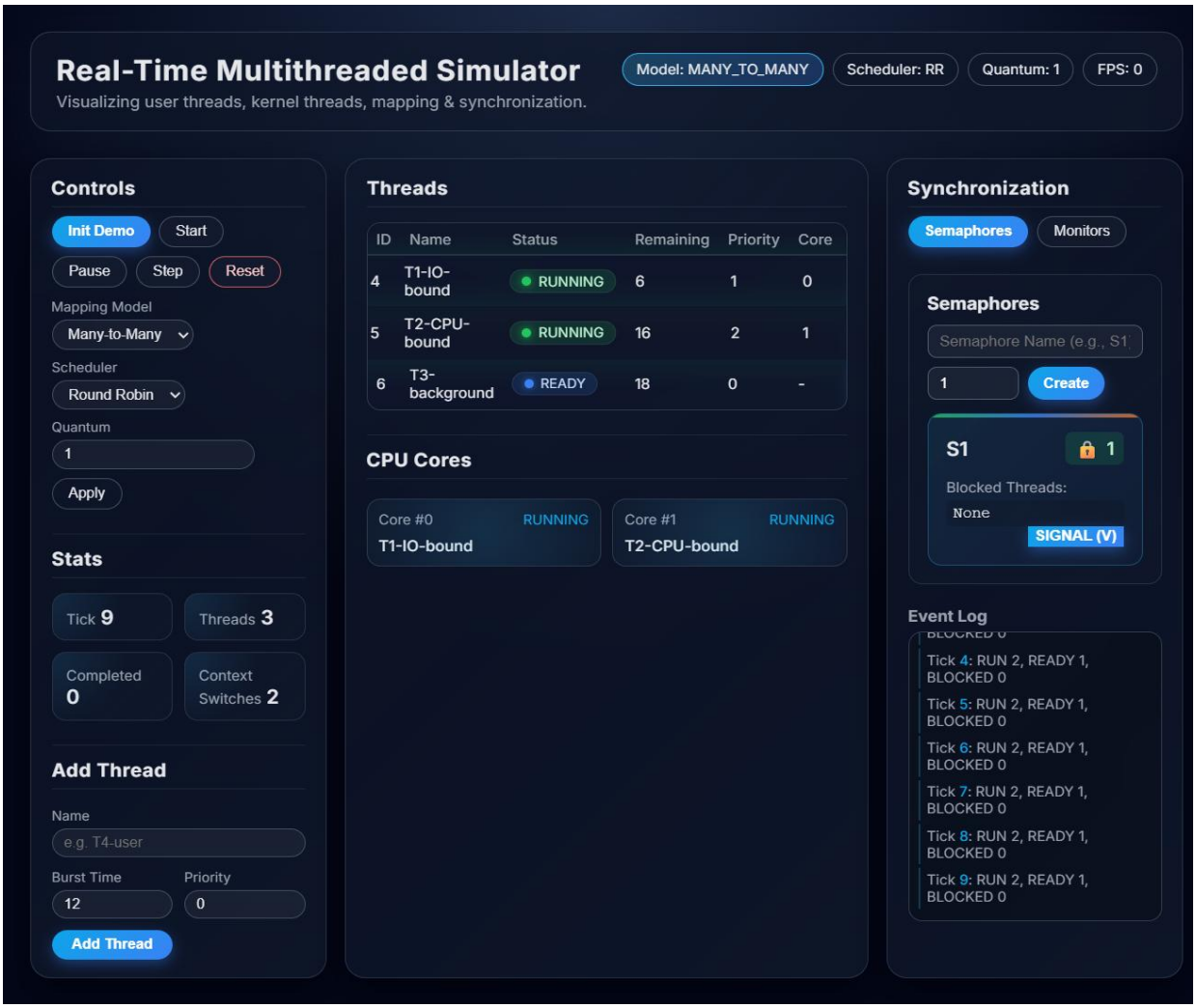
1. Enhanced Synchronization Mechanisms
    - Addition of read–write locks, barriers, cyclic barriers, and latch mechanisms.
    - More complex monitor behaviors and conditional event chaining.

2. Performance Analytics
    - Graphical representation of metrics such as waiting time, turnaround time, response time, and CPU utilization.
    - Generation of Gantt charts for detailed CPU scheduling visualization.

Overall, these enhancements would elevate the simulator from an academic demonstration tool to a fully featured OS education and experimentation platform, capable of supporting more advanced research, teaching, and system analysis.

8.REFERENCE

]


Real-Time Multithreaded Simulator
Visualizing user threads, kernel threads, mapping & synchronization.
Model: MANY_TO_MANY
Scheduler: RR
Quantum: 1
FPS: 0
Controls
Init Demo
Start
Pause
Step
Reset
Mapping Model
Many-to-Many
Scheduler
Round Robin
Quantum
1
Apply
Stats
Tick 9
Threads 3
Completed 0
Context Switches 2
Add Thread
Name
e.g. T4-user
Burst Time
12
Priority
0
Add Thread
Threads
ID Name Status Remaining Priority Core
4 T1-IO-bound RUNNING 6 1 0
5 T2-CPU-bound RUNNING 16 2 1
6 T3-background READY 18 0 -
CPU Cores
Core #0 RUNNING
T1-IO-bound
Core #1 RUNNING
T2-CPU-bound
Synchronization
Semaphores
Monitors
Semaphores
Semaphore Name (e.g., S1
1
Create
S1
1
Blocked Threads:
None
SIGNAL (V)
Event Log
Tick 4: RUN 2, READY 1, BLOCKED 0
Tick 5: RUN 2, READY 1, BLOCKED 0
Tick 6: RUN 2, READY 1, BLOCKED 0
Tick 7: RUN 2, READY 1, BLOCKED 0
Tick 8: RUN 2, READY 1, BLOCKED 0
Tick 9: RUN 2, READY 1, BLOCKED 0

Project structure:

```
├── .gitignore
├── README.md
├── requirements.txt
├── src
    ├── __init__.py
    ├── app.py
    └── simulator
    |    ├── __init__.py
    |    ├── controller.py
    |    ├── core.py
    |    └── sync.py
├── static
    ├── css
    |    └── style.css
    └── js
    |    └── app.js
└── templates
    └── index.html
```

Controller.py

```
"""
SimulationController:
- Manages simulated threads, kernel threads, schedulers, mapping models
- Runs the ticking loop (background thread)
- Provides functions used by the Flask API (including semaphore & monitor ops)
"""

import threading
import time
from typing import Dict, List, Optional

from .core import (
   SimThread,
```

```
    KernelThread,
    ThreadState,
    RoundRobinScheduler,
    FCFSScheduler,
    PriorityScheduler,
    MappingModel,
)
from .sync import SimSemaphore, Monitor

TICK_INTERVAL = 1

class SimulationController:
    """
    Central manager for the simulator.
    """

    def __init__(self, num_kernel_threads: int = 2,
                 model: MappingModel =
MappingModel.MANY_TO_MANY,
                 scheduler_name: str = "RR",
                 quantum: int = 1):

        self.lock = threading.RLock()

        self.threads: Dict[int, SimThread] = {}

        self.kernels: List[KernelThread] = [KernelThread(i) for i in
range(num_kernel_threads)]

        self.model = model
        self.quantum = quantum
        self.scheduler = self._make_scheduler(scheduler_name)
```

```
        self.running = False
        self._tick = 0
        self._runner_thread: Optional[threading.Thread] = None

        self.semaphores: Dict[str, SimSemaphore] = {}
        self.monitors: Dict[str, Monitor] = {}

        self.stats = {"context_switches": 0, "completed": 0}

    def _make_scheduler(self, name: str):
        name = (name or "RR").upper()
        if name in ("RR", "ROUNDROBIN"):
            return RoundRobinScheduler()
        if name in ("FCFS", "FIFO"):
            return FCFSScheduler()
        if name in ("PRIORITY", "PR"):
            return PriorityScheduler()
        return RoundRobinScheduler()

    def set_scheduler(self, name: str):
        """
        Change scheduler type and re-register all non-terminated,
non-blocked threads
        so the new scheduler gets a proper READY queue.
        """
        with self.lock:
            new_sched = self._make_scheduler(name)

            for t in self.threads.values():
                if t.state not in (ThreadState.TERMINATED,
ThreadState.BLOCKED):
                    new_sched.add(t)
```

```
            self.scheduler = new_sched

    def set_model(self, model_name: str):
        with self.lock:
            if isinstance(model_name, str):
                self.model = MappingModel[model_name]
            elif isinstance(model_name, MappingModel):
                self.model = model_name

    def set_quantum(self, q: int):
        with self.lock:
            self.quantum = int(q)

    def add_thread(self, total_burst: int = 10, priority: int = 0,
name: str = None) -> SimThread:
        with self.lock:
            t = SimThread(total_burst=total_burst, priority=priority,
name=name)
            t.state = ThreadState.READY
            self.threads[t.id] = t
            self.scheduler.add(t)
            return t

    def get_thread(self, tid: int) -> Optional[SimThread]:
        return self.threads.get(int(tid))

    def create_semaphore(self, name: str, initial: int = 1) ->
SimSemaphore:
        with self.lock:
            s = SimSemaphore(initial, name=name)
            self.semaphores[name] = s
            return s
```

```
    def get_semaphore(self, name: str) ->
Optional[SimSemaphore]:
        return self.semaphores.get(name)

    def create_monitor(self, name: str) -> Monitor:
        with self.lock:
            m = Monitor()
            self.monitors[name] = m
            return m

    def get_monitor(self, name: str) -> Optional[Monitor]:
        return self.monitors.get(name)

    def semaphore_wait(self, sem_name: str, thread_id: int) ->
Dict:
        """
        Attempt P() on semaphore for given thread.
        If the semaphore causes blocking, mark thread BLOCKED
and remove from scheduler.
        Returns a dict: {"acquired": bool, "blocked": bool}
        """
        with self.lock:
            sem = self.semaphores.get(sem_name)
            t = self.threads.get(int(thread_id))
            if sem is None:
                return {"ok": False, "error": f"Semaphore '{sem_name}'
not found."}
            if t is None:
                return {"ok": False, "error": f"Thread id {thread_id} not
found."}

            acquired = sem.P(t)
            if not acquired:
```

```
                t.state = ThreadState.BLOCKED
                try:
                    self.scheduler.remove(t)
                except Exception:
                    pass
                for k in self.kernels:
                    if k.current_thread is t:
                        k.release()
                return {"ok": True, "acquired": False, "blocked": True}
            else:
                return {"ok": True, "acquired": True, "blocked": False}

    def semaphore_signal(self, sem_name: str) -> Dict:
        """
        Perform V() on semaphore; if an unblocked thread returned
by V(), set it to READY and re-add to scheduler.
        """
        with self.lock:
            sem = self.semaphores.get(sem_name)
            if sem is None:
                return {"ok": False, "error": f"Semaphore '{sem_name}'
not found."}
            unblocked = sem.V()
            if unblocked is not None:
                unblocked.state = ThreadState.READY
                self.scheduler.add(unblocked)
                return {"ok": True, "unblocked": unblocked.id}
            return {"ok": True, "unblocked": None}

    def monitor_wait(self, monitor_name: str, cond_name: str,
thread_id: int) -> Dict:
        with self.lock:
            mon = self.monitors.get(monitor_name)
```

```
            t = self.threads.get(int(thread_id))
            if mon is None:
                return {"ok": False, "error": f"Monitor
'{monitor_name}' not found."}
            if t is None:
                return {"ok": False, "error": f"Thread id {thread_id} not
found."}
            t.state = ThreadState.BLOCKED
            mon.wait(cond_name, t)
            try:
                self.scheduler.remove(t)
            except Exception:
                pass
            for k in self.kernels:
                if k.current_thread is t:
                    k.release()
            return {"ok": True, "blocked": t.id}

    def monitor_signal(self, monitor_name: str, cond_name: str) -
> Dict:
        with self.lock:
            mon = self.monitors.get(monitor_name)
            if mon is None:
                return {"ok": False, "error": f"Monitor
'{monitor_name}' not found."}
            th = mon.signal(cond_name)
            if th:
                th.state = ThreadState.READY
                self.scheduler.add(th)
                return {"ok": True, "unblocked": th.id}
            return {"ok": True, "unblocked": None}
```

```
    def monitor_broadcast(self, monitor_name: str, cond_name:
str) -> Dict:
        with self.lock:
            mon = self.monitors.get(monitor_name)
            if mon is None:
                return {"ok": False, "error": f"Monitor
'{monitor_name}' not found."}
            lst = mon.broadcast(cond_name)
            for th in lst:
                th.state = ThreadState.READY
                self.scheduler.add(th)
            return {"ok": True, "unblocked": [t.id for t in lst]}

    def get_state_snapshot(self) -> Dict:
        with self.lock:
            threads = [
                {
                    "id": t.id,
                    "name": t.name,
                    "state": t.state.value,
                    "remaining": t.remaining,
                    "priority": t.priority,
                    "mapped_kernel": t.mapped_kernel,
                }
                for t in sorted(self.threads.values(), key=lambda x: x.id)
            ]
            kernels = [
                {
                    "id": k.id,
                    "current_thread": k.current_thread.id if
k.current_thread else None,
                }
                for k in self.kernels
```

```
            ]
            sems = {name: {"value": s.value, "blocked": [th.id for th in
s.peek_blocked()]} for name, s in self.semaphores.items()}
            mons = {name: {"conds": {}} for name in
self.monitors.keys()}
            stats = dict(self.stats)
            return {
                "tick": self._tick,
                "threads": threads,
                "kernels": kernels,
                "semaphores": sems,
                "monitors": mons,
                "model": self.model.value,
                "quantum": self.quantum,
                "stats": stats,
            }

    def _assign_thread_to_kernel(self, thread: SimThread) ->
KernelThread:
        if self.model == MappingModel.MANY_TO_ONE:
            k = self.kernels[0]
            k.assign(thread)
            return k
        elif self.model == MappingModel.ONE_TO_ONE:
            idx = (thread.id - 1) % len(self.kernels)
            k = self.kernels[idx]
            k.assign(thread)
            return k
        else:
            for k in self.kernels:
                if k.current_thread is None or k.current_thread.state in
(ThreadState.TERMINATED, ThreadState.BLOCKED):
                    k.assign(thread)
```

```
                return k
        k = self.kernels[0]
        k.assign(thread)
        return k

    def _run_tick(self):
        with self.lock:
            self._tick += 1

        for k in self.kernels:

            if k.current_thread is None or k.current_thread.state in
(
                ThreadState.TERMINATED,
                ThreadState.BLOCKED,
            ):
                nxt = self.scheduler.next()
                if nxt is None:
                    k.release()
                    continue
                if nxt.state == ThreadState.TERMINATED:
                    continue

                self.stats["context_switches"] += 1
                nxt.state = ThreadState.RUNNING
                k.assign(nxt)

            if k.current_thread is None:
                continue

            used = k.current_thread.run_slice(self.quantum)

            if k.current_thread.state == ThreadState.TERMINATED:
```

```
                self.stats["completed"] += 1
                k.release()
                continue




    def start(self):
        with self.lock:
            if self.running:
                return
            self.running = True
            self._runner_thread =
threading.Thread(target=self._run_loop, daemon=True)
            self._runner_thread.start()

    def _run_loop(self):
        while True:
            with self.lock:
                if not self.running:
                    break
            self._run_tick()
            time.sleep(TICK_INTERVAL)

    def pause(self):
        with self.lock:
            self.running = False

    def step(self):
        with self.lock:
            self._run_tick()

    def reset(self):
```

```
        with self.lock:
            self.threads.clear()
            for k in self.kernels:
                k.release()
            self.semaphores.clear()
            self.monitors.clear()
            self._tick = 0
            self.stats = {"context_switches": 0, "completed": 0}
            self.scheduler = self._make_scheduler("RR")
```

App.py

```
"""
Flask API for the Multithreaded Simulator.
Frontend (HTML/CSS/JS) will communicate with this server to:
- Get state
- Add threads
- Control simulation (start/pause/step/reset)
- Manage semaphores & monitors
"""

from flask import Flask, jsonify, request, send_from_directory
import os

from src.simulator.controller import SimulationController

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
TEMPLATES_DIR = os.path.join(BASE_DIR, "..", "templates")
STATIC_DIR = os.path.join(BASE_DIR, "..", "static")
```

```
app = Flask(__name__, static_folder=STATIC_DIR,
template_folder=TEMPLATES_DIR)

controller = SimulationController(num_kernel_threads=2)

def init_demo_state():
    """
    Reset the simulator and create some default demo threads +
a semaphore,
    then start the simulation.
    Call this once (e.g. from frontend on page load).
    """
    controller.reset()

    controller.set_model("MANY_TO_MANY")
    controller.set_scheduler("RR")
    controller.set_quantum(1)

    controller.add_thread(total_burst=15, priority=1, name="T1-
IO-bound")
    controller.add_thread(total_burst=25, priority=2, name="T2-
CPU-bound")
    controller.add_thread(total_burst=18, priority=0, name="T3-
background")

    controller.create_semaphore("S1", initial=1)

    controller.start()

@app.route("/api/state", methods=["GET"])
def api_state():
    return jsonify(controller.get_state_snapshot())
```

```
@app.route("/api/start", methods=["POST"])
def api_start():
    controller.start()
    return jsonify({"ok": True})

@app.route("/api/pause", methods=["POST"])
def api_pause():
    controller.pause()
    return jsonify({"ok": True})

@app.route("/api/step", methods=["POST"])
def api_step():
    controller.step()
    return jsonify({"ok": True})

@app.route("/api/reset", methods=["POST"])
def api_reset():
    controller.reset()
    return jsonify({"ok": True})

@app.route("/api/add_thread", methods=["POST"])
def api_add_thread():
    data = request.json or {}
    burst = int(data.get("burst", 10))
    pr = int(data.get("priority", 0))
    name = data.get("name")
    t = controller.add_thread(total_burst=burst, priority=pr,
name=name)
    return jsonify({"ok": True, "thread": {"id": t.id, "name":
t.name}})

@app.route("/api/set", methods=["POST"])
def api_set():
```

```
    data = request.json or {}
    if "model" in data:
        try:
            controller.set_model(data["model"])
        except Exception as e:
            return jsonify({"ok": False, "error": str(e)}), 400
    if "scheduler" in data:
        controller.set_scheduler(data["scheduler"])
    if "quantum" in data:
        controller.set_quantum(int(data["quantum"]))
    return jsonify({"ok": True})

@app.route("/api/init_demo", methods=["POST"])
def api_init_demo():
    """
    Initialize a default demo scenario:
    - Reset everything
    - Create demo threads & semaphore
    - Start simulation
    """
    init_demo_state()
    return jsonify({"ok": True})

@app.route("/api/semaphore/create", methods=["POST"])
def api_sem_create():
    data = request.json or {}
    name = data.get("name")
    initial = int(data.get("initial", 1))
    if not name:
        return jsonify({"ok": False, "error": "name required"}), 400
    s = controller.create_semaphore(name, initial)
    return jsonify({"ok": True, "semaphore": {"name": s.name,
"value": s.value}})
```

```
@app.route("/api/semaphore/wait", methods=["POST"])
def api_sem_wait():
    data = request.json or {}
    name = data.get("name")
    tid = data.get("thread_id")
    if not name or tid is None:
        return jsonify({"ok": False, "error": "name and thread_id
required"}), 400
    result = controller.semaphore_wait(name, tid)
    return jsonify(result)

@app.route("/api/semaphore/signal", methods=["POST"])
def api_sem_signal():
    data = request.json or {}
    name = data.get("name")
    if not name:
        return jsonify({"ok": False, "error": "name required"}), 400
    result = controller.semaphore_signal(name)
    return jsonify(result)

@app.route("/api/monitor/create", methods=["POST"])
def api_mon_create():
    data = request.json or {}
    name = data.get("name")
    if not name:
        return jsonify({"ok": False, "error": "name required"}), 400
    m = controller.create_monitor(name)
    return jsonify({"ok": True})

@app.route("/api/monitor/wait", methods=["POST"])
def api_mon_wait():
    data = request.json or {}
```

```
    name = data.get("name")
    cond = data.get("cond", "default")
    tid = data.get("thread_id")
    if not name or tid is None:
        return jsonify({"ok": False, "error": "name and thread_id
required"}), 400
    result = controller.monitor_wait(name, cond, tid)
    return jsonify(result)

@app.route("/api/monitor/signal", methods=["POST"])
def api_mon_signal():
    data = request.json or {}
    name = data.get("name")
    cond = data.get("cond", "default")
    if not name:
        return jsonify({"ok": False, "error": "name required"}), 400
    result = controller.monitor_signal(name, cond)
    return jsonify(result)

@app.route("/", methods=["GET"])
def index():
    return send_from_directory(TEMPLATES_DIR, "index.html")

@app.route("/static/<path:path>")
def serve_static(path):
    return send_from_directory(STATIC_DIR, path)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT",
5000)), debug=True)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Multithreaded Simulator Dashboard</title>
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <link rel="stylesheet" href="/static/css/style.css" />

  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com"
crossorigin />
  <link
    href="https://fonts.googleapis.com/css2?family=Inter:wght@
300;400;500;600;700&display=swap"
    rel="stylesheet"
  />
</head>

<body>
<div class="app-root">

  <header class="app-header glass">
    <div class="header-main">
      <h1>Real-Time Multithreaded Simulator</h1>
      <p class="subtitle">
        Visualizing user threads, kernel threads, mapping &
synchronization.
      </p>
    </div>
```

```
    <div class="header-badges">
      <span class="badge badge-primary" id="badge-
model">Model: -</span>
      <span class="badge badge-secondary" id="badge-
scheduler">Scheduler: RR</span>
      <span class="badge badge-secondary" id="badge-
quantum">Quantum: 1</span>
      <span class="badge badge-secondary" id="badge-fps">FPS:
0</span>
    </div>
  </header>

  <main class="layout-grid">

    <section class="panel glass" id="control-panel">

      <div class="panel-header">
        <h2>Controls</h2>
      </div>

      <div class="controls-grid">
        <div class="controls-row">
          <button class="btn btn-primary" id="btn-init-demo">Init
Demo</button>
          <button class="btn" id="btn-start">Start</button>
          <button class="btn" id="btn-pause">Pause</button>
          <button class="btn" id="btn-step">Step</button>
          <button class="btn btn-danger" id="btn-
reset">Reset</button>
        </div>

        <div class="controls-row">
```

```
        <div class="field-group">
          <label for="select-model">Mapping Model</label>
          <select id="select-model">
            <option value="MANY_TO_ONE">Many-to-
One</option>
            <option value="ONE_TO_ONE">One-to-One</option>
            <option value="MANY_TO_MANY" selected>Many-to-
Many</option>
          </select>
        </div>

        <div class="field-group">
          <label for="select-scheduler">Scheduler</label>
          <select id="select-scheduler">
            <option value="RR">Round Robin</option>
            <option value="FCFS">FCFS</option>
            <option value="PRIORITY">Priority</option>
          </select>
        </div>

        <div class="field-group">
          <label for="input-quantum">Quantum</label>
          <input id="input-quantum" type="number" min="1"
value="1" />
        </div>

        <button class="btn btn-outline" id="btn-apply-
config">Apply</button>
      </div>
    </div>

    <div class="panel-header panel-header--spaced">
      <h2>Stats</h2>
```

```
    </div>

    <div class="stats-grid">
     <div class="stat-card glass-soft">
      <span class="stat-label">Tick</span>
      <span class="stat-value" id="stat-tick">0</span>
     </div>

     <div class="stat-card glass-soft">
      <span class="stat-label">Threads</span>
      <span class="stat-value" id="stat-threads">0</span>
     </div>

     <div class="stat-card glass-soft">
      <span class="stat-label">Completed</span>
      <span class="stat-value" id="stat-completed">0</span>
     </div>

     <div class="stat-card glass-soft">
      <span class="stat-label">Context Switches</span>
      <span class="stat-value" id="stat-context-
switches">0</span>
     </div>
    </div>

    <div class="panel-header panel-header--spaced">
     <h2>Add Thread</h2>
    </div>

    <form id="form-add-thread" class="add-thread-form">
     <div class="field-group">
      <label for="thread-name">Name</label>
```

```
        <input id="thread-name" type="text" placeholder="e.g.
T4-user" />
      </div>

      <div class="field-group">
        <label for="thread-burst">Burst Time</label>
        <input id="thread-burst" type="number" min="1"
value="12" />
      </div>

      <div class="field-group">
        <label for="thread-priority">Priority</label>
        <input id="thread-priority" type="number" min="0"
value="0" />
      </div>

      <button class="btn btn-primary" type="submit">Add
Thread</button>
    </form>

  </section>

  <section class="panel glass" id="threads-panel">
    <div class="panel-header">
      <h2>Threads</h2>
    </div>

    <div class="threads-table-wrapper">
      <table class="threads-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
```

```
            <th>Status</th>
            <th>Remaining</th>
            <th>Priority</th>
            <th>Core</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody id="threads-tbody"></tbody>
      </table>
    </div>

    <div class="panel-header panel-header--spaced">
      <h2>CPU Cores</h2>
    </div>

    <div class="cores-grid" id="cores-grid"></div>
  </section>

  <section class="panel glass" id="sync-panel">

    <div class="panel-header">
      <h2>Synchronization</h2>
    </div>

    <div class="controls-row" style="margin-bottom:12px;">
      <button id="btn-tab-semaphores" class="btn btn-
primary">Semaphores</button>
      <button id="btn-tab-monitors"
class="btn">Monitors</button>
    </div>

    <div id="sync-panel-semaphores">
```

```
      <div class="card semaphore-panel glass-soft">
        <h2 class="semaphore-title">Semaphores</h2>

        <div class="semaphore-form-row">
          <input id="sem-name" class="input"
placeholder="Semaphore Name (e.g., S1)" />
          <input id="sem-init" class="input input-small"
type="number" value="1" />
          <button onclick="createSemaphore()" class="btn btn-
primary">Create</button>
        </div>

        <div id="sem-list" class="semaphore-list"></div>
      </div>

    </div>

    <div id="sync-panel-monitors" style="display:none;">

      <div class="card semaphore-panel glass-soft">
        <h2 class="semaphore-title">Monitors</h2>

        <div class="semaphore-form-row">
          <input id="mon-name" class="input"
placeholder="Monitor Name (e.g., M1)" />
          <button onclick="createMonitor()" class="btn btn-
primary">Create</button>
        </div>

        <div id="mon-list" class="semaphore-list"></div>
      </div>

    </div>
```

```
      <div class="sync-section">
        <h3>Event Log</h3>
        <div id="event-log" class="event-log"></div>
      </div>

    </section>

  </main>
</div>

<script src="/static/js/app.js" defer></script>
</body>
</html>
```

App.js

```
const API = {
  state: "/api/state",
  start: "/api/start",
  pause: "/api/pause",
  step: "/api/step",
  reset: "/api/reset",
  initDemo: "/api/init_demo",
  addThread: "/api/add_thread",
  setConfig: "/api/set",

  semCreate: "/api/semaphore/create",
  semWait: "/api/semaphore/wait",
  semSignal: "/api/semaphore/signal",

  monCreate: "/api/monitor/create",
  monWait: "/api/monitor/wait",
```

```
  monSignal: "/api/monitor/signal",
  monBroadcast: "/api/monitor/broadcast",
};

let pollTimer = null;
let lastTick = 0;

let currentSyncMode = "semaphore";

function createSemaphore() {
  const name = document.getElementById("sem-
name").value.trim();
  const init = parseInt(document.getElementById("sem-
init").value, 10);

  if (!name) return alert("Enter semaphore name.");

  fetch(API.semCreate, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, initial: init }),
  }).then(refreshState);
}

function semaphoreWait(semName, threadId) {
  fetch(API.semWait, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: semName, thread_id: threadId
}),
  }).then(refreshState);
}
```

```
function semaphoreSignal(semName) {
  fetch(API.semSignal, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: semName }),
  }).then(refreshState);
}

function createMonitor() {
  const name = document.getElementById("mon-
name").value.trim();
  if (!name) return alert("Enter monitor name.");

  fetch(API.monCreate, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name }),
  }).then(refreshState);
}

function monitorWait(monitor, threadId) {
  fetch(API.monWait, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ monitor, thread_id: threadId, cond:
"default" }),
  }).then(refreshState);
}

function monitorSignal(monitor) {
  fetch(API.monSignal, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
```

```
    body: JSON.stringify({ monitor, cond: "default" }),
  }).then(refreshState);
}

function monitorBroadcast(monitor) {
  fetch(API.monBroadcast, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ monitor, cond: "default" }),
  }).then(refreshState);
}

async function apiPost(url, body) {
  const res = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: body ? JSON.stringify(body) : null,
  });
  return res.json();
}

async function apiGet(url) {
  const res = await fetch(url);
  return res.json();
}

function makeThreadActions(thread, semaphores, monitors) {
  if (thread.state === "TERMINATED") return "-";
  if (thread.state === "BLOCKED") return "<em>BLOCKED</em>";

  let html = "";

  if (currentSyncMode === "semaphore") {
```

```
    for (let name of Object.keys(semaphores)) {
      html += `
        <button class="btn-primary"
onclick="semaphoreWait('${name}', ${thread.id})">
          WAIT ${name}
        </button>
      `;
    }
  }

  if (currentSyncMode === "monitor") {
    for (let name of Object.keys(monitors)) {
      html += `
        <button class="btn-primary"
onclick="monitorWait('${name}', ${thread.id})">
          WAIT ${name}
        </button>
      `;
    }
  }

  return html || "<em>No actions</em>";
}

function renderState(state) {
  document.getElementById("stat-tick").textContent = state.tick;
  document.getElementById("stat-threads").textContent =
state.threads.length;
  document.getElementById("stat-completed").textContent =
state.stats.completed;
  document.getElementById("stat-context-
switches").textContent =
    state.stats.context_switches;
```

```
  document.getElementById("badge-model").textContent =
`Model: ${state.model}`;
  document.getElementById("badge-quantum").textContent =
`Quantum: ${state.quantum}`;

  const tbody = document.getElementById("threads-tbody");
  tbody.innerHTML = "";

  state.threads.forEach((t) => {
    const tr = document.createElement("tr");

    const running = state.kernels.some((k) => k.current_thread
=== t.id);
    if (running) tr.classList.add("thread-row-running");

    const stateClass =
      t.state === "RUNNING" ? "state-running" :
      t.state === "READY" ? "state-ready" :
      t.state === "BLOCKED" ? "state-blocked" :
      "state-terminated";

    const stateCell = `
      <div class="thread-state-pill ${stateClass}">
        <span class="thread-state-dot"></span>
        <span>${t.state}</span>
      </div>
    `;

    tr.innerHTML = `
      <td>${t.id}</td>
      <td>${t.name}</td>
      <td>${stateCell}</td>
```

```
      <td>${t.remaining}</td>
      <td>${t.priority}</td>
      <td>${t.mapped_kernel ?? "-"}</td>
      <td>${makeThreadActions(t, state.semaphores,
state.monitors)}</td>
    `;

    tbody.appendChild(tr);
  });

  const coresGrid = document.getElementById("cores-grid");
  coresGrid.innerHTML = "";

  state.kernels.forEach((k) => {
    const div = document.createElement("div");
    const busy = k.current_thread !== null;

    const threadObj = state.threads.find((t) => t.id ===
k.current_thread);

    div.className = "core-card" + (busy ? " busy" : "");
    div.innerHTML = `
      <div class="core-pulse"></div>
      <div class="core-header">
        <span class="core-name">Core #${k.id}</span>
        <span class="core-status ${busy ? "core-status-busy" :
"core-status-idle"}">
          ${busy ? "RUNNING" : "IDLE"}
        </span>
      </div>
      <div class="core-body">
        ${
          busy && threadObj
```

```
          ? `<div class="core-thread-
name">${threadObj.name}</div>`
          : `<div class="core-thread-name" style="color:var(--
muted);">No thread</div>`
        }
      </div>
    `;
    coresGrid.appendChild(div);
  });

  const semList = document.getElementById("sem-list");
  semList.innerHTML = "";

  for (let [name, sem] of Object.entries(state.semaphores)) {
    const div = document.createElement("div");
    div.className = "sem-card";

    div.innerHTML = `
      <div class="sem-header">
        <span class="sem-name">${name}</span>
        <span class="sem-value ${sem.value > 0 ? "available" :
"locked"}">
          <span class="value-icon">🔒</span> ${sem.value}
        </span>
      </div>

      <div class="sem-blocked">
        <div class="blocked-label">Blocked Threads:</div>
        <div class="blocked-value">${sem.blocked.join(", ") ||
"None"}</div>
      </div>

      <div class="sem-actions">
```

```
      <button class="btn-primary"
onclick="semaphoreSignal('${name}')">SIGNAL (V)</button>
    </div>
   `;

   semList.appendChild(div);
  }

  const monList = document.getElementById("mon-list");
  if (monList) {
   monList.innerHTML = "";

   for (let [name, mon] of Object.entries(state.monitors)) {
    const div = document.createElement("div");
    div.className = "sem-card";

    div.innerHTML = `
     <div class="sem-header">
      <span class="sem-name">${name}</span>
     </div>

     <div class="sem-blocked">
      <div class="blocked-label">Waiting:</div>
      <div class="blocked-value">${mon.waiting.join(", ") ||
"None"}</div>
     </div>

     <div class="sem-actions">
      <button class="btn-primary"
onclick="monitorSignal('${name}')">SIGNAL</button>
      <button class="btn-primary"
onclick="monitorBroadcast('${name}')">BROADCAST</button>
     </div>
```

```
      `;
      monList.appendChild(div);
    }
  }

  if (state.tick !== lastTick) {
    const log = document.getElementById("event-log");
    const line = document.createElement("div");
    line.className = "event-line";

    const running = state.threads.filter((t) => t.state ===
"RUNNING").length;
    const ready = state.threads.filter((t) => t.state ===
"READY").length;
    const blocked = state.threads.filter((t) => t.state ===
"BLOCKED").length;

    line.innerHTML = `Tick <span>${state.tick}</span>: RUN
${running}, READY ${ready}, BLOCKED ${blocked}`;
    log.appendChild(line);
    log.scrollTop = log.scrollHeight;

    lastTick = state.tick;
  }
}

async function pollState() {
  try {
    const state = await apiGet(API.state);
    renderState(state);
  } catch (err) {
    console.error("Error fetching state", err);
  }
```

```
}

function startPolling() {
  if (pollTimer) clearInterval(pollTimer);
  pollTimer = setInterval(pollState, 500);
}

function setupControls() {
  document.getElementById("btn-init-
demo").addEventListener("click", async () => {
    await apiPost(API.initDemo);
    lastTick = 0;
    document.getElementById("event-log").innerHTML = "";
    pollState();
  });

  document.getElementById("btn-
start").addEventListener("click", async () => {
    await apiPost(API.start);
    document.body.classList.remove("no-animations");
  });

  document.getElementById("btn-
pause").addEventListener("click", async () => {
    await apiPost(API.pause);
    document.body.classList.add("no-animations");
  });

  document.getElementById("btn-
step").addEventListener("click", async () => {
    await apiPost(API.step);
    pollState();
  });
```

```
  document.getElementById("btn-
reset").addEventListener("click", async () => {
    await apiPost(API.reset);
    lastTick = 0;
    document.getElementById("event-log").innerHTML = "";
    pollState();
  });

  document.getElementById("btn-apply-
config").addEventListener("click", async () => {
    const model = document.getElementById("select-
model").value;
    const scheduler = document.getElementById("select-
scheduler").value;
    const quantum = parseInt(document.getElementById("input-
quantum").value, 10);

    await apiPost(API.setConfig, { model, scheduler, quantum });

    document.getElementById("badge-scheduler").textContent =
`Scheduler: ${scheduler}`;
    document.getElementById("badge-quantum").textContent =
`Quantum: ${quantum}`;
  });

  document.getElementById("form-add-
thread").addEventListener("submit", async (e) => {
    e.preventDefault();

    const name = document.getElementById("thread-
name").value.trim() || null;
```

```
    const burst = parseInt(document.getElementById("thread-
burst").value, 10);
    const priority = parseInt(document.getElementById("thread-
priority").value, 10);

    await apiPost(API.addThread, { name, burst, priority });

    document.getElementById("thread-name").value = "";
    pollState();
  });

  document.getElementById("btn-tab-semaphores").onclick = ()
=> {
    currentSyncMode = "semaphore";
    document.getElementById("sync-panel-
semaphores").style.display = "block";
    document.getElementById("sync-panel-
monitors").style.display = "none";
  };

  document.getElementById("btn-tab-monitors").onclick = () => {
    currentSyncMode = "monitor";
    document.getElementById("sync-panel-
semaphores").style.display = "none";
    document.getElementById("sync-panel-
monitors").style.display = "block";
  };
}

window.addEventListener("DOMContentLoaded", async () => {
  setupControls();
  try {
    await apiPost(API.initDemo);
```

```
  } catch (err) {
    console.error("Failed to init demo", err);
  }
  startPolling();
});
```